

EXERCISE I

In reference to the techniques used to reduce the computational effort in CNN

I.a) Describe the concept of pointwise convolution

Pointwise convolution, often referred to as 1x1 convolution, is a technique used in Convolutional Neural Networks (CNNs) where the filter size is 1x1. Unlike traditional convolutions that operate over spatial dimensions (e.g., 3x3 or 5x5), pointwise convolutions operate on the depth dimension of the input tensor. In a pointwise convolution, each 1x1 filter is applied to every pixel across all channels of the input tensor, producing a single output value per channel.

Pointwise convolutions are usually exploited either to reduce the number of channels (dimensionality reduction) or to increase it (dimensionality expansion).

I.b) Write a formula to estimate the number of MAC operations for a pointwise convolution with N input channels (128x64 size) to produce 32 output channels

$$\text{MAC} = \text{Cin} * \text{Cout} * \text{Wout} * \text{Hout} * K * K = N * 32 * 128 * 64 * 1 * 1 \quad \text{MAC} = N * 262144 \quad \text{MAC}$$

I.c) How many weights do you need to store for a depth-wise + pointwise convolution with 5x5 filters, 32x32 sized channels, and 64 input channels, 128 output channels. Assume padding with zeros.

Depthwise convolution (DW), Pointwise convolution (PW)

$$\text{Params_dw} = K * K * \text{Cin} = 5 * 5 * 64 = 1600$$

$$\text{Params_pw} = K * K * \text{Cin} * \text{Cout} = 1 * 1 * 64 * 128 = 8192$$

$$\text{Tot} = 1600 + 8192 = 9792$$

EXERCISE II

Consider a convolutional network module with the following structure: 32 input channels (size 64x64), first convolutional layer (3x3 filters) with 32 output channels followed by max-pooling (size 2x2, stride 2), cascaded with a second layer with depth-wise (5x5 filters) + pointwise convolutions with 128 output channels and RELU non-linearity. Assume padding with zeros for the convolutions.

II.a) Compute the number of MAC operations needed to compute the first convolutional layer.

Layers:

(Win, Hin, Cout) -->

Input --> Conv1 (3x3) --> Max-pool 2x2 --> DW_Conv2 (5x5) --> PW_Conv3 (1x1) --> ReLU
64*64*32 --> 64*64*32 --> 32*32*32 --> 32*32*32 --> 32*32*128 --> 32*32*128

$$\text{MAC_conv1} = \text{Cin} * \text{Cout} * \text{Wout} * \text{Hout} * K * K = 32 * 32 * 64 * 64 * 3 * 3 = 37748736$$

II.b) Compute the total number of MAC operations needed to compute all the output channels for the three convolutions.

$$\text{MAC_conv1} = \text{Cin} * \text{Cout} * \text{Wout} * \text{Hout} * K * K = 32 * 32 * 64 * 64 * 3 * 3 = 37748736$$

$$\text{MAC_DW_conv2} = \text{Cout} * \text{Wout} * \text{Hout} * K * K = 32 * 32 * 32 * 5 * 5 = 819200$$

$$\text{MAC_PW_conv3} = \text{Cin} * \text{Cout} * \text{Wout} * \text{Hout} * K * K = 32 * 128 * 32 * 32 * 1 * 1 = 4194304$$

$$\text{Tot} = 37748736 + 819200 + 4194304 = 42762240$$

II.c) Compute the number of operations involved in max-pooling and RELU (assume one max-pooling operator requires 5 operations and one RELU operator requires 3 operations)

Operations:

- COMPARE = 5 ops
- Relu = 3 ops

$$\begin{aligned} \text{MAXPOOL_ops} &= (2*2-1) \text{ COMPARE} * \text{Cout} * \text{Wout} * \text{Hout} = (3 \text{ COMPARE}) * 32 * 32 * 32 = \\ &= (3 * 5 \text{ ops}) * 32768 = 491520 \text{ ops} \end{aligned}$$

$$\text{ReLU_ops} = 32 * 32 * 128 * \text{RELU} = 131072 * 3 \text{ops} = 393216 \text{ ops}$$

EXERCISE III

A parallel cluster of RISC-V processors is executing a matrix multiplication of matrices A (32x64) and B (128x32). Assume that the computation is parallelized assigning tiles of size 8x8 of the output matrix C to processors, following a row-major tile order. The processor cluster has 10 processors. Elements of A, B, C are 16bit FP numbers

III.a) Compute the L1 minimal shared memory size that is needed to store the complete matrices, and the total number of MACs needed to complete the matrix multiplication

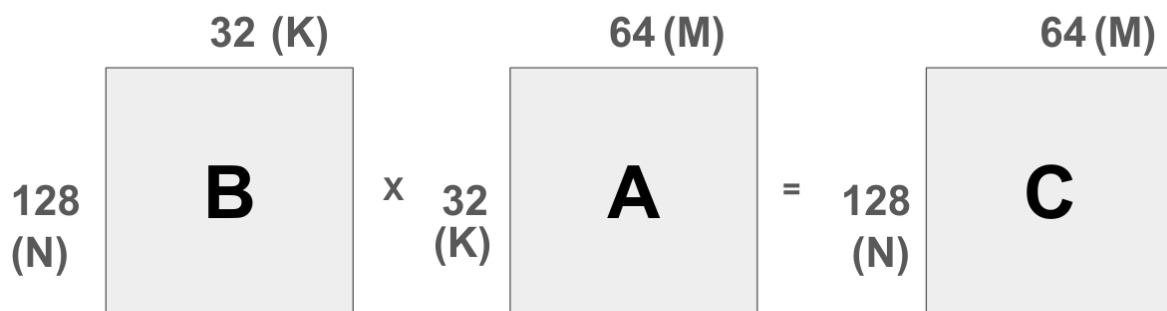
$$A = 32 \times 64$$

$$B = 128 \times 32$$

$$C = A \times B = 128 \times 64$$

$$N_{\text{cores}} = 10$$

$$\text{Precision} = \text{FP16}$$



$$N^{\circ} \text{ Tiles (N)} = 64/8 = 8$$

$$N^{\circ} \text{ Tiles (M)} = 128/8 = 16$$

Memory:

$$\text{Precision} = \text{FP16} = 2B$$

$$\text{Mem (A)} = 128 \times 32 \times 2B$$

$$\text{Mem(B)} = 32 \times 64 \times 2B$$

$$\text{Mem (C)} = 64 \times 128 \times 2B$$

$$\text{Mem (A,B,C)} = 28672 B$$

MAC:

$$\text{MAC (AxB)} = 128 \times 32 \times 64 = 262144 \text{ MAC}$$

III.b) Compute the number of tiles to be assigned to each core, trying to minimize the workload unbalance. If the number is not identical for all the cores, give min and max.

$$N^{\circ} \text{ Tiles (N)} = 64/8 = 8$$

$$N^{\circ} \text{ Tiles (M)} = 128/8 = 16$$

$$N^{\circ} \text{ Tiles (tot)} = 8 * 16 = 128$$

Number of tiles assigned to 10 cores

$$N^{\circ} \text{ Tiles(to each core)} = 128/10 \text{ -- The workload is unbalanced}$$

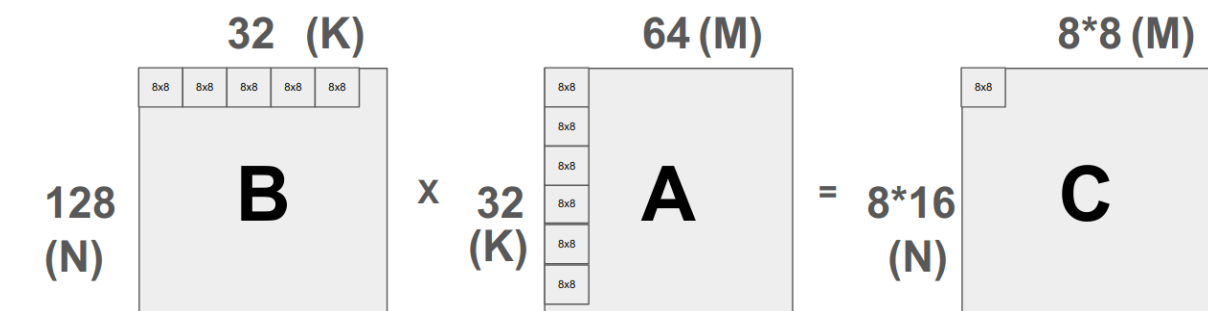
To minimize the workload unbalance, out of 10 cores:

- 8 cores will process 13 tiles
- 2 cores will process 12 tiles

III.c) Compute the speedup with respect to sequential computation. Assume 15% of stall cycles due to banking conflicts.

$$\text{Ops (tot)} = 262144 \text{ MAC}$$

As all the cores process the tiles in parallel, the total number of operations is split among all the cores. The worst case is when a core has to process 13 tiles, instead of 12, therefore:



$$\text{Ops (1 output tile)} = 8 * 8 * 8 * 32/4 = 2048 \text{ MAC}$$

$$\text{Ops (13 output tile)} = \text{Ops (1 output tile)} * 13 = 2048 \text{ MAC} * 13 = 26624 \text{ MAC}$$

$$\text{Ops (13 output tile, 15\% stall)} = \text{ceil}(\text{Ops (1 output tile)} * 13 * 1.15) = \text{ceil}(2048 \text{ MAC} * 13 * 1.15) = 30618 \text{ MAC}$$

$$\text{Speedup} = 262144 / 30618 = 8.56 \%$$

Or, more simply

$$\text{Speedup} = 128 / (13 * 1.15) = 8.56 \%$$